

# Git & Javascript basic

## Lesson 2

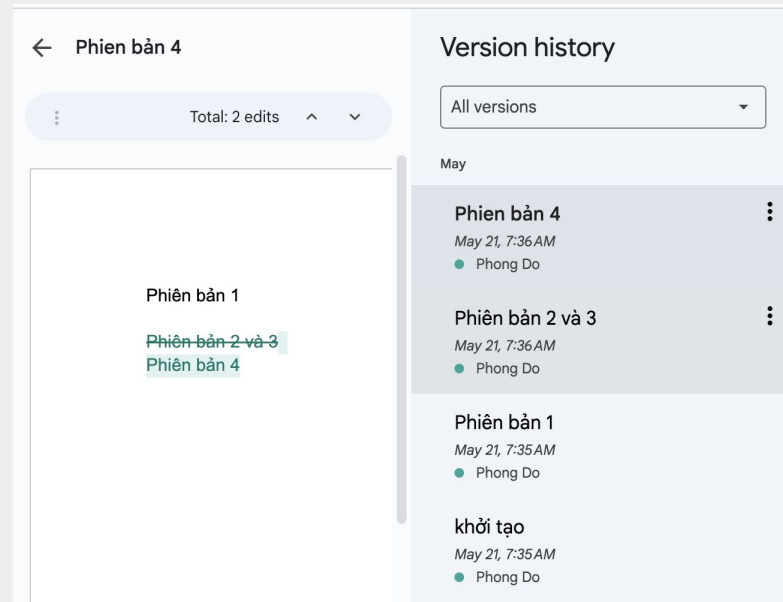


# Review homework

# Agenda

1. Version control system
2. Git
3. Javascript basic

# Version control system



# Version Control System

- VCS (**Version Control System**): Hệ thống quản lý các phiên bản
  - **Google Docs** sử dụng Version Control System để dễ quản lý các phiên bản

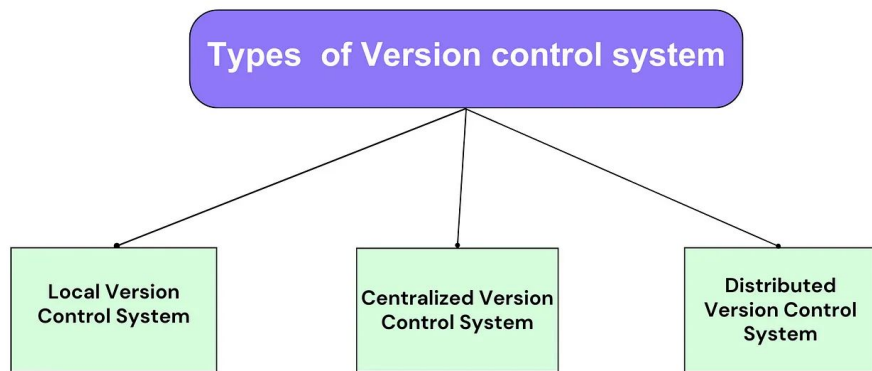
The screenshot displays the 'Version history' sidebar in Google Docs. The main document area shows 'Phiên bản 4' selected. The sidebar lists the following versions:

| Version          | Time            | Editor   |
|------------------|-----------------|----------|
| Phiên bản 4      | May 21, 7:36 AM | Phong Do |
| Phiên bản 2 và 3 | May 21, 7:36 AM | Phong Do |
| Phiên bản 1      | May 21, 7:35 AM | Phong Do |
| khởi tạo         | May 21, 7:35 AM | Phong Do |

At the top of the sidebar, a dropdown menu is set to 'All versions'. The main document area also shows a summary: 'Phiên bản 4' with 'Total: 2 edits'.

# Version Control System

- **Local:** lưu ở máy cá nhân
- **Centralize:** lưu ở một máy chủ tập trung.
- **Distributed:** lưu ở nhiều máy khác nhau

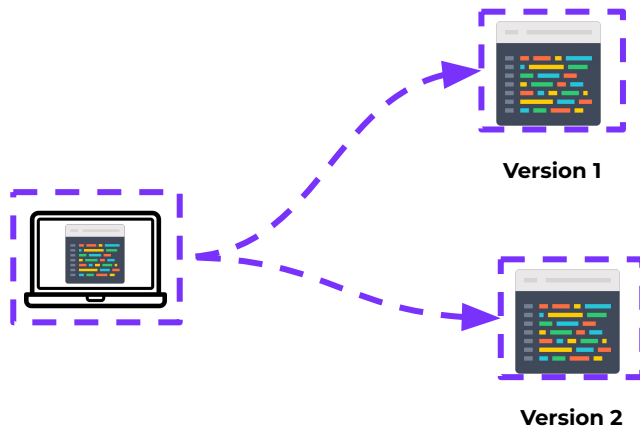


**Git**



# Git

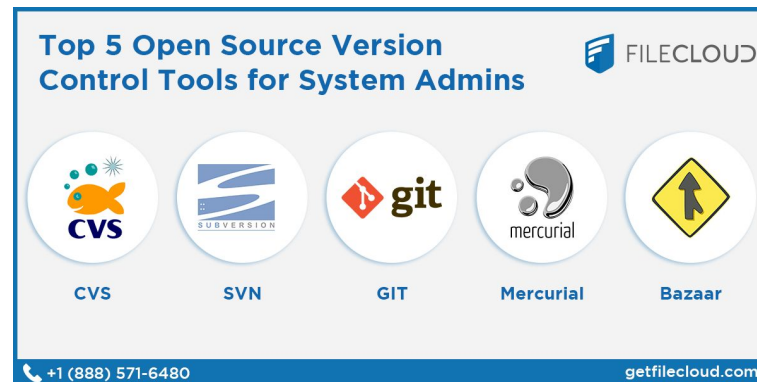
- **Git?** Tại sao dùng Git
  - **Git** được viết bởi **Linus Torvalds**, cha đẻ của Linux
  - **git** là từ viết **sai chính tả** (có chủ đích) của get, do get đã được dùng rồi
  - Dùng git do **nhu cầu quản lý phiên bản** và **làm việc giữa nhiều người với nhau**





# Git & other VCS

- So sánh Git với các VCS khác
  - **Tính năng:** Dễ dùng, có nhiều tính năng vượt trội: branching, tốc độ xử lý nhanh
  - **Chi phí:** Free
  - **Phổ biến:** nhiều công ty sử dụng



# Git & GitHub

| Git                                                       | GitHub                                                               |
|-----------------------------------------------------------|----------------------------------------------------------------------|
| Là một phần mềm                                           | Là một dịch vụ web                                                   |
| Cài trên máy của bạn                                      | Host trên website                                                    |
| Là một commandline tool                                   | Là công cụ có giao diện                                              |
| Là công cụ quản lý phiên bản, đưa file vào Git repository | Là nơi để upload Git repository lên                                  |
| Có các tính năng của Version Control System               | Có các tính năng của Version Control System và một số tính năng khác |

# Git - three states

## Working Directory

Các file mới hoặc file  
có thay đổi

## Staging Area

Các file đưa vào  
vùng chuẩn bị  
commit (tạo ra  
các phiên bản)

## Repository

Các commit  
(phiên bản)

# Git



`test1.spec.ts`

`.gitignore`

`package.json`

# Git status



**git init**

# Git status

Working Directory



`test1.spec.ts`

`.gitignore`

`package.json`

# Git status

Working Directory



test1.spec.ts

.gitignore

package.json

Staging Area



# Git status

Working Directory

Staging Area



test1.spec.ts

.gitignore

package.json

```
git add test1.spec.ts
```

test1.spec.ts





# Git status

Working Directory



test1.spec.ts

.gitignore

package.json

Staging Area

test1.spec.ts

# Git status

Working Directory



test1.spec.ts

.gitignore

package.json

Staging Area

test1.spec.ts

Repository

test1.spec.ts

`git commit -m "feat: add test 1"`

# Git status

Working Directory



test1.spec.ts

.gitignore

package.json

Staging Area

Repository

test1.spec.ts

“feat: add ...”

`test1.spec.ts`

# Git status

Working Directory

`.gitignore``package.json`

Staging Area

`git add .`

`.gitignore``package.json`

Repository

`test1.spec.ts``"feat: add ..."`

# Git status

Working Directory



test1.spec.ts

.gitignore

package.json

Staging Area

```
git commit -m "chore: add other files"
```

.gitignore

package.json

Repository

test1.spec.ts

"feat: add ..."

.gitignore

package.json

## Working Directory

`test1.spec.ts``.gitignore``package.json`

## Staging Area

## Repository

`test1.spec.ts``“feat: add ...”``.gitignore``package.json``“chore: ...”`

# Git - key takeaways

- Câu lệnh thường dùng:
  - Khởi tạo thư mục được quản lý bởi Git
    - `git init`

```
> git init
Initialized empty Git repository in /Users/apple/work/tmp.nosync/git-example/.git/
```

# Git - key takeaways

- Câu lệnh thường dùng
  - Khởi tạo
  - Cấu hình
- Sau khi cài đặt xong, cần cấu hình git:
  - Cho 1 repo
    - `git config user.name "<name>"`
    - `git config user.email "<email>"`
  - Cho toàn bộ máy tính (default)
    - `git config --global user.name "user"`
    - `git config --global user.email "email"`



# Git - key takeaways

- Câu lệnh thường dùng
  - Khởi tạo
  - Cấu hình
- Thực hành
  - Tạo 2 thư mục: `lesson-2-git1`, `lesson2-git2`
  - Khởi tạo git và thực hiện cấu hình
    - `lesson-2-git1`: `name="Jame", email="jame@playwrightvn.com"`
    - `lesson-2-git2`: `name="Alex", email="alex@playwrightvn.com"`

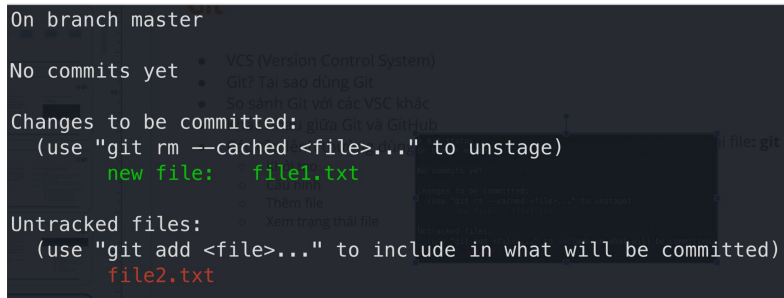
# Git

- Câu lệnh thường dùng
  - Khởi tạo
  - Cấu hình
  - Thêm file vào vùng staging

```
> git add file1.txt  
> git add .
```

- Thêm file vào vùng staging:
  - Thêm 1 file: `git add <file_name>`
  - Thêm toàn bộ: `git add .`
- Thực hành:
  - `lesson-2-git1`:
    - Thêm files: 01.txt, 02.txt
    - Chạy lệnh: `git add 01.txt`
  - `lesson-2-git2`:
    - Thêm files: 03.txt, 04.txt
    - Chạy lệnh: `git add .`

- Câu lệnh thường dùng
  - Khởi tạo
  - Cấu hình
  - Thêm file vào vùng staging
  - Xem trạng thái file



- Xem trạng thái file: **git status**
  - File **màu xanh**: vùng **staging**
  - File **màu đỏ**: vùng **working directory**
- Thực hành:
  - Chạy lệnh **git status** trên repo **lesson-2-git1** và **lesson-2-git2**
  - Liệt kê các file ở vùng **working directory** và vùng **repository**

# Git

- Câu lệnh thường dùng
  - Khởi tạo
  - Cấu hình
  - Thêm file vào vùng staging
  - Commit
- Commit: `git commit -m "message"`
- Kiểm tra lịch sử commit: `git log`

# Git - commit convention

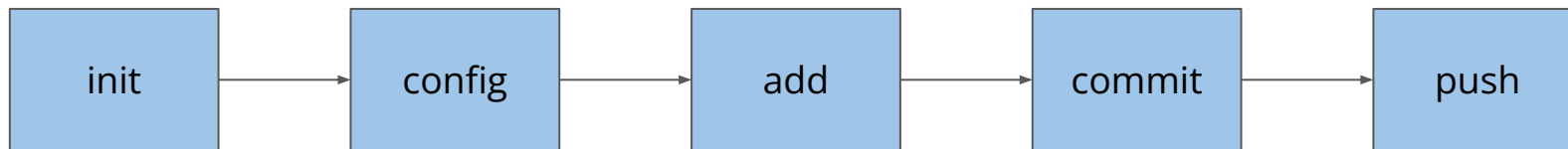
- Commit convention?
- Convention = quy tắc
- Convention giúp:
  - Cả team làm việc với nhau theo một quy tắc
  - Code gọn gàng, sạch đẹp hơn

# Git - commit convention

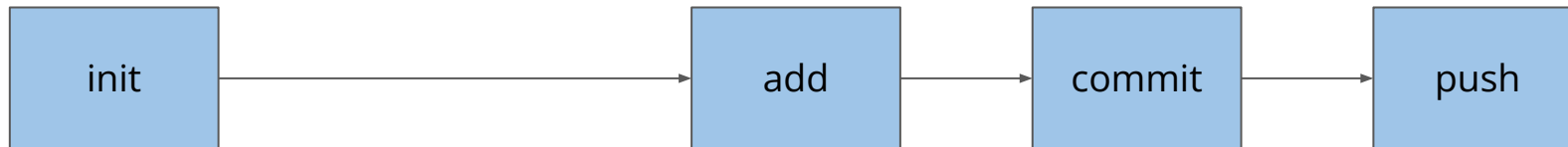
- Commit convention?
  - Convention sử dụng
- Commit message cần rõ ràng, chuyên nghiệp
  - Trong lớp học, dùng convention sau:  
**<type>: <short\_description>**
  - Trong đó:
    - type: loại commit
      - **chore**: sửa nhỏ lẻ, chỉnh tả, xóa file không dùng tới,...
      - **feat**: thêm tính năng mới, test case mới
      - **fix**: sửa lỗi 1 test trước đó
    - short\_description: mô tả ngắn gọn (50 kí tự), tiếng Anh hoặc tiếng Việt không dấu.
  - Ví dụ
    - "chore: remove unused file"
    - "feat: add code for exercises 2"

# Git - simple workflow

## Không dùng global config



## Dùng global config



# Javascript basic





# Javascript

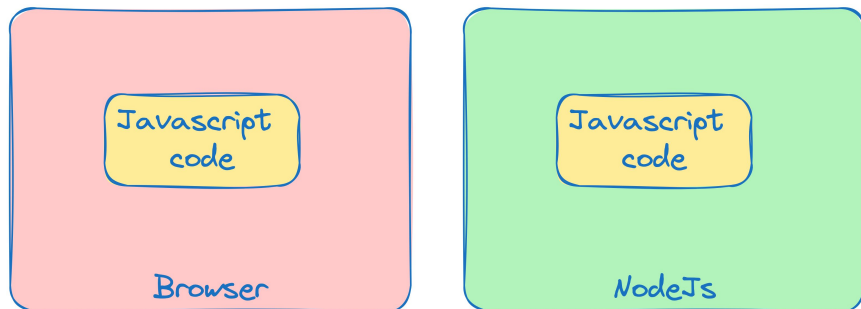
- Javascript?
  - Fun fact: “Ăn theo” cái tên hot hit: Java



- Là một **ngôn ngữ lập trình**.
- Ra đời năm 1995 bởi **Brendan Eich**.
- Giúp cho browser hoạt động được.
- Top language: [Stackoverflow report](#)

# Javascript

- Javascript?



- Bình thường Javascript chạy được do browser engine support
  - **Edge:** Chakra
  - **Firefox:** SpiderMonkey
  - **Chrome:** V8
- Để chạy được trên máy tính không cần trình duyệt, cần Node Js
- run-time

# Javascript

- Hello world
- Tạo thư mục  
lesson-2/javascript
- Mở bằng VS Code: `code .`
- Tạo file: **01-hello.js**
- Ghi vào dòng:  
`console.log("Hello World!");`
- Chạy lệnh: `node  
lesson-2/javascript/01-hello.js`

# Javascript

- Hello world
- Variable
  - 02-variable.js

Variable = biến, dùng để lưu trữ giá trị, có thể thay đổi giá trị được.

- Khai báo:
  - `var <ten_bien> = <gia tri>;`
  - `let <ten_bien> = <gia tri>;`
  - Trong đó: `<ten_bien>`
    - Bắt đầu bằng ký tự chữ, hoặc gạch dưới, hoặc \$
    - KHÔNG chứa dấu cách
    - KHÔNG là các "từ khóa", tức là các từ đã có trong Javascript (ví dụ: `var`, `let`, `for`, `if`,...)

# Javascript

- Hello world
- Variable
  - 02-variable.js

Variable = biến, dùng để lưu trữ giá trị, có thể thay đổi giá trị được.

- Khai báo:
- Ví dụ:
  - **var firstName = "Playwright";**
  - **let lastName = "Automation class";**

# Javascript

- Hello world
- Variable
  - 02-variable.js

Variable = biến, dùng để lưu trữ giá trị, có thể thay đổi giá trị được.

- Khai báo:
- Ví dụ:
- Sử dụng:
  - `console.log("firstname = " + firstName);`
  - `console.log("lastName = " + lastName);`

# Javascript

- Hello world
- Variable
  - 02-variable.js

Variable = biến, dùng để lưu trữ giá trị, có thể thay đổi giá trị được.

- Khai báo:
- Ví dụ:
- Sử dụng:
- Gán lại
  - **lastName = "Lesson 2";**

# Javascript

- Hello world
- Variable
  - 02-variable.js

Variable = biến, dùng để lưu trữ giá trị, có thể thay đổi giá trị được.

- Khai báo:
- Ví dụ:
- Sử dụng:
- Kiểm tra
  - `console.log("lastName = " + lastName);`



# Javascript

- Hello world
- Variable
  - 02-variable.js
- Thực hành với học sinh:  
Khai báo các biến sau:
  - Tên biến: **name**, giá trị: "**<Tên của bạn>**"  
(Lưu ý, có ngoặc kép "")
  - Tên biến: **age**, giá trị: **<tuổi của bạn>**

# Javascript

- Hello world
- Variable
  - 02-variable.js

Khác nhau giữa **var** và **let**?

- **var** khai báo lại được, **let** thì không
  - `var item1 = 'a';`
  - `var item1 = 'c'; // OK`
  - `let item2 = 'b';`
  - `let item2 = 'd'; // error`
- Phạm vi của biến
  - **var**: phạm vi toàn cục (global)
  - **let**: phạm vi trong cặp ngoặc `{ }`
- **Nên dùng**: let, vì dễ kiểm soát phạm vi của biến
- Chi tiết sẽ học ở **"Bài 3"**
- **Thực hành**: Khai báo lại các biến ở ví dụ trước dùng let.

# Javascript

- Hello world
- Variable, constant

**Constant** = **hằng số**. Dùng để khai báo các giá trị không thể thay đổi.

- **Khai báo**
  - `const <name> = <value>`
- **Ví dụ**
  - `const framework = "Playwright";`
- **Sử dụng**
  - `console.log(framework);`
- **Gán lại**
  - `framework = "Cypress" // Error`
- **Thực hành:** Khai báo các hằng số sau:
  - Tên hằng số: **name**, giá trị: "<Tên của bạn>" (Lưu ý, có ngoặc kép "")
  - Tên hằng số: **age**, giá trị: <tuổi của bạn>

# Javascript

- Hello world
- Variable, constant

Khi nào dùng var/let, khi nào dùng const?

- **var/let**: khi biến sẽ **gán lại**
- **const**: khi biến **không gán lại**
- Thường sẽ dùng **let** và **const**, KHÔNG dùng **var**

# Javascript

- Hello world
- Variable, constant, data types

Data type = kiểu dữ liệu.

- Có 8 loại kiểu dữ liệu: **String**, **Number**, **BigInt**, **Boolean**, **Undefined**, **Null**, **Symbol**, **Object**.
- Ở các bài đầu, chúng ta sẽ quan tâm đến các kiểu: **String**, **Number**, **Boolean**.
  - Kiểu **String**: dùng để khai báo một **chuỗi**
    - Ví dụ: "Phong", "Playwright class", "Playwright Việt Nam"
  - Kiểu **Number**: dùng để khai báo **một số**.
    - Ví dụ: 100, 0, 100.5, -100.005
  - Kiểu **Boolean**: dùng để khai báo một giá trị **kiểu đúng sai** (true hoặc false):
    - Ví dụ: true, false.

# Javascript

- Hello world
- Variable, constant, data types
- **Thực hành:**
  - Khai báo các biến có kiểu dữ liệu sau:
  - Tên biến: **name**, kiểu dữ liệu: **string**, giá trị: **tên bạn**
  - Tên biến: **isLearningPlaywright**, kiểu dữ liệu: **boolean**, giá trị: **đúng**.
  - Tên biến: **price**, kiểu dữ liệu: **number**, giá trị: **200**.

# Javascript

- Hello world
- Variable, constant, data types
- Comparison operator

Comparison operator = toán tử so sánh.

- Dùng để **so sánh giá trị** giữa 2 biến với nhau. Kết quả sẽ trả về **Boolean** (true hoặc false).
- Các toán tử so sánh:
  - So sánh hơn kém: **>, <**
  - So sánh bằng: **==, ===, !=, !==, >=, <=**
- Ở bài này, tập trung dùng so sánh hơn kém và **một vài** so sánh bằng (**===, !==, >=, <=**)

# Javascript

- Hello world repo
- Variable, constant, data types
- Comparison operator
- Unary operator

**Unary operator** = toán tử một ngôi.

- Dùng để tăng hoặc giảm giá trị
- **`i++`** bằng với **`i=i+1`**
- **`i--`** bằng với **`i=i-1`**



# Javascript

- Hello world repo
- Variable, constant, data types
- Comparison operator
- Arithmetic operator

**Arithmetic operator** = toán tử số học.

- Dùng tính toán giá trị biểu thức
- Các phép toán: **+**, **-**, **\***, **/**

# Javascript

- Hello world repo
- Variable, constant, data types
- Comparison operator
- Conditional

**Conditional** = điều kiện, dùng để kiểm tra có nên thực hiện một đoạn logic không.

- Cú pháp: `if (<điều kiện>) { // code }`. Nếu điều kiện đúng, sẽ chạy đoạn code
- Ví dụ:

```
if (5 > 3) {  
    console.log("5 lớn hơn 3")  
}
```

- Thực hành:

# Javascript

- Hello world repo
- Variable, constant, data types
- Comparison operator
- Conditional
- Loops

Loops = vòng lặp

- Dùng để thực hiện một đoạn logic một số lần nhất định
- Cú pháp: `for(<khởi tạo>; <điều kiện dừng>; <điều kiện tăng>) {  
// code }`
- Ví dụ:

```
for (let i = 1; i <= 10; i++) {  
    console.log("Giá trị của i là: ", i);  
}
```

- Thực hành

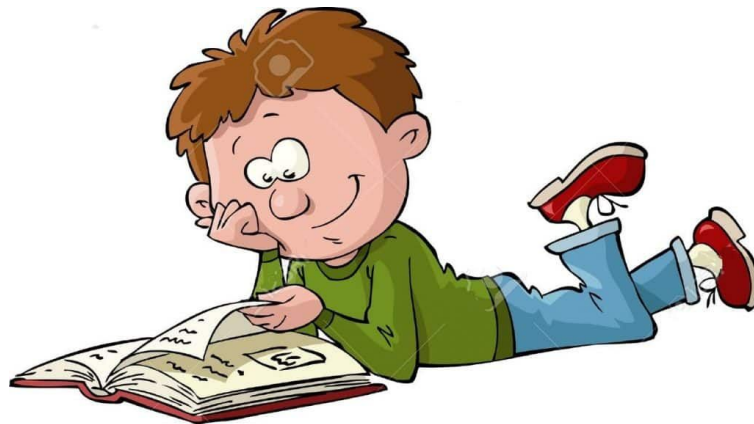
# VSCode

- Format code
  - Mac: Option + Shift + F
  - Window: Alt + Shift + F



# Key takeaways

- Từ bài học 2, bạn hãy thử tổng hợp kiến thức đã được học sau mỗi buổi học bằng file markdown.
- Có thêm mục “Kiến thức bổ sung để làm bài”



# Homework

# What's next

- Git & Javascript basic (continue)
  - **Git**
    - Undo things
    - Branching
  - **Javascript**
    - Scope
    - == and !=
    - Advance condition
    - Advance loops
    - Object, Array, Function

# Thank you

